

Census Income Classification: Understanding Income Inequality

Isuri Nawodya Herath

Northeastern University

ALY 6020: Predictive Analytics

Census Income Classification: Income Inequality

Income inequality remains one of the most pressing policy challenges in the United States. Organizations working to ensure equal pay require data-driven tools to understand which attributes correlate with higher earning potential and where disparities may exist. This paper presents the development of a K-Nearest Neighbors (KNN) classification model trained on U.S. Census data to distinguish citizens earning above \$50,000 annually from those earning at or below that threshold. The analysis proceeds in two parts: a data cleansing pipeline that prepares the dataset for modeling, and a KNN modeling process that produces, evaluates, and interprets results for a stakeholder audience.

Part 1: Data Cleansing

Dataset Overview

The dataset used in this analysis is derived from the UCI Adult Census repository and contains 42,468 records after preprocessing. Each record represents an individual U.S. citizen and includes 14 attributes: age, workclass, fnlwgt, education, education-num, marital-status, occupation, relationship, race, sex, capital-gain, capital-loss, hours-per-week, and native-country. The dependent variable is salary, coded as $\leq 50K$ or $> 50K$. The continuous variables are age, education-num, capital-gain, capital-loss, and hours-per-week. All remaining variables are categorical.

Cleansing Process

Step 1: Loading and Initial Inspection

The dataset was loaded with column names assigned manually, as the raw file contains no header row. Missing values in this dataset are encoded as the string "?". To address this at load time, the parameter `na_values="?"` was passed during import so that all such entries were converted to NaN immediately. A subsequent missing value audit confirmed that no null values were present across any column, meaning no imputation or row removal was required. This finding was documented explicitly to satisfy the data quality audit trail.

Step 2: Dropping Irrelevant Features

The column `fnlwgt` (final weight) is a Census Bureau sampling weight used to extrapolate survey results to the broader population. It carries no predictive value for individual income classification and was explicitly identified in the dataset description as a column to ignore. It was therefore dropped before any further analysis.

Additionally, `education` and `education-num` were found to encode identical information, with `education` being the categorical label and `education-num` being its ordinal numeric equivalent. Retaining both would have effectively double-weighted education in the KNN distance calculation. The categorical `education` column was dropped and the numeric `education-num` was retained, as KNN requires all features to be numeric.

Step 3: Duplicate Removal

A duplicate row check was performed using pandas' `duplicated()` method. Duplicate records were identified and removed to prevent any single observation from exerting disproportionate influence on model training. The final dataset retained 42,468 unique records after this step.

Step 4: Outlier Detection and Treatment

Outlier detection was performed using the interquartile range (IQR) method. Any value falling below $Q1 - 1.5 \times IQR$ or above $Q3 + 1.5 \times IQR$ was flagged as an outlier. The results showed that capital-gain contained 3,997 outliers (9.4%) and capital-loss contained 2,265 outliers (5.3%), both driven by extreme right skew where the majority of individuals have a value of zero but a small number report very large figure. hours-per-week had 9,260 flagged values (21.8%); however, this was not treated as a data quality problem because non-standard working hours reflect a legitimate real-world distribution rather than recording errors.

```
continuous_cols = ['age', 'education-num', 'capital-gain', 'capital-loss', 'hours-per-week']

for col in continuous_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    outlier_count = ((df[col] < lower) | (df[col] > upper)).sum()
    print(f"{col}: {outlier_count} outliers ({outlier_count/len(df)*100:.1f}%)")

age: 184 outliers (0.4%)
education-num: 323 outliers (0.8%)
capital-gain: 3997 outliers (9.4%)
capital-loss: 2265 outliers (5.3%)
hours-per-week: 9260 outliers (21.8%)
```

For capital-gain and capital-loss, a 99th percentile capping strategy was applied. Values above the 99th percentile were clipped to that threshold, preserving all rows while reducing the influence of extreme values on Euclidean distance calculations. After capping, the maximum value of capital-gain dropped from 99,999 to 15,024 and capital-loss dropped from 4,356 to 2,002. Rows were not removed because these values, while extreme, represent real financial activity and removing them would have introduced selection bias.

Step 5: Encoding

The target variable salary was converted from string labels to binary integers: 0 for $\leq 50K$ and 1 for $> 50K$. A class distribution check revealed that approximately 75.4% of records fall in the $\leq 50K$ category and 24.6% in the $> 50K$ category, indicating a meaningful class imbalance. This imbalance informed the decision to use stratified train/test splitting and to prioritize F1-score alongside accuracy as evaluation metrics.

One-hot encoding was applied to all nominal categorical variables: workclass, marital-status, occupation, relationship, race, sex, and native-country. The parameter `drop_first=True` was used to remove one category per variable, avoiding the dummy variable trap in which one dummy column is perfectly predictable from the others. Following encoding, the feature space expanded from 13 columns to 84 predictor columns.

Step 6: Feature Scaling and Train/Test Split

StandardScaler normalization was applied to all 84 predictor features. This step is essential for KNN because the algorithm relies entirely on Euclidean distance to measure similarity between observations. Without scaling, variables with large numeric ranges, such as capital-gain with a maximum of 15,024, would dominate the distance calculation and effectively suppress the contribution of variables with smaller ranges, such as education-num which spans only 1 to 16. After scaling, all features have a mean of zero and a standard deviation of one.

The dataset was split into 80% training (33,974 records) and 20% test (8,494 records) sets using `random_state=42` for reproducibility. The `stratify=y` parameter was applied to ensure that the 75/25 class ratio was preserved in both the training and test sets, preventing the model from being evaluated on an unrepresentative test distribution.

Part 2: KNN Model Development and Results

K-Nearest Neighbors: Conceptual Overview

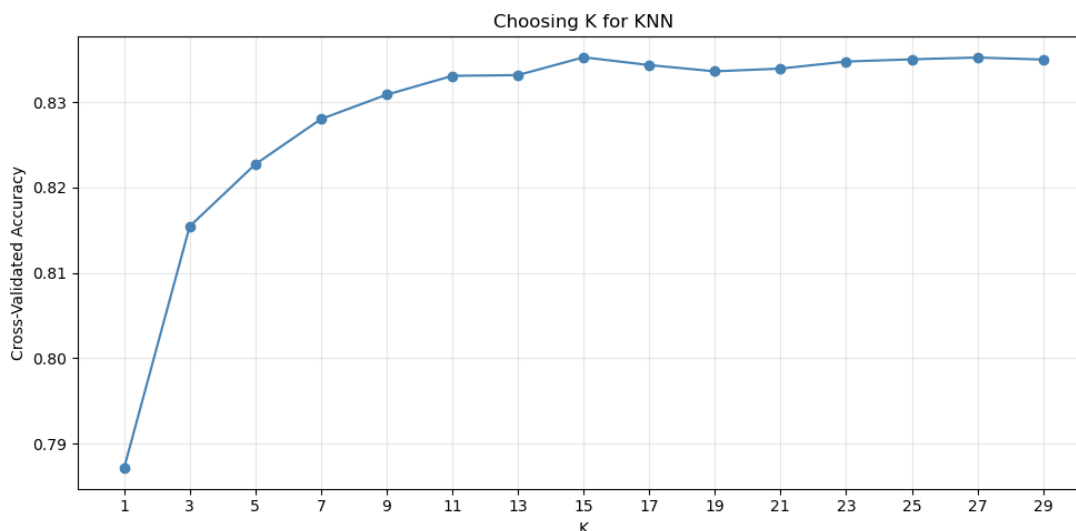
The K-Nearest Neighbors algorithm classifies a new observation by identifying the K training observations closest to it in feature space, then assigning the class that appears most frequently among those neighbors. The algorithm has no explicit training phase: it memorizes the training data and performs all computation at prediction time by computing distances between the query point and every stored observation. By default, sklearn's `KNeighborsClassifier` uses Euclidean distance as the proximity measure. The predicted class is determined by a majority vote among the K neighbors, and class probabilities can be estimated as the proportion of neighbors belonging to each class.

Determining the Optimal K

The choice of K is the primary hyperparameter governing KNN performance. A low K value, such as $K = 1$, causes the model to memorize individual training points and become highly sensitive to noise, resulting in overfitting. A high K value causes the model to consider too broad a neighborhood, smoothing the decision boundary excessively and potentially underfitting by ignoring meaningful local patterns.

To determine the optimal K, 5-fold cross-validation was applied across odd values of K from 1 to 30. Odd values were selected specifically to prevent ties in majority voting for binary classification. The mean cross-validated

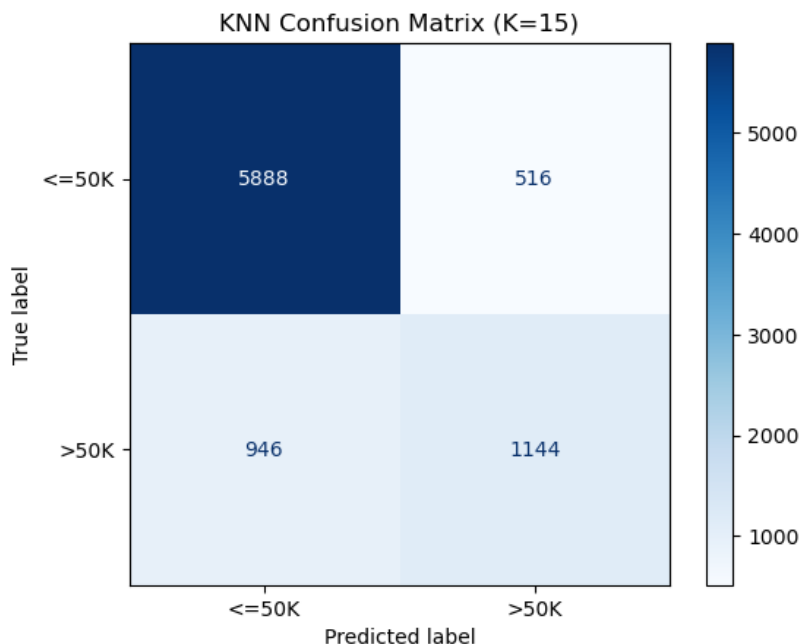
accuracy was recorded for each K value and plotted against K to produce an elbow curve. The elbow point, where accuracy stabilizes and further increases in K produce diminishing returns, was identified at $K = 15$. At this value, the model achieves the best balance between bias and variance. $K = 15$ was therefore selected as the final hyperparameter.



Model Performance

The final KNN model with $K = 15$ was trained on the full training set and evaluated on the held-out test set. Overall accuracy was 83%, meaning the model correctly classified approximately 83 out of every 100 citizens. The AUC score was 0.8748, indicating strong discriminative ability between high and low income earners.

The classification report revealed a precision of 0.69 and recall of 0.55 for the $>50K$ class, yielding an F1-score of 0.61. The lower recall for the $>50K$ class means the model misses approximately 45% of actual high earners, classifying them incorrectly as low income. For the $\leq 50K$ class, precision was 0.86 and recall was 0.92, reflecting considerably stronger performance on the majority class. This disparity is a direct consequence of the 75/25 class imbalance, where the model has far more examples of low-income citizens to learn from during training.

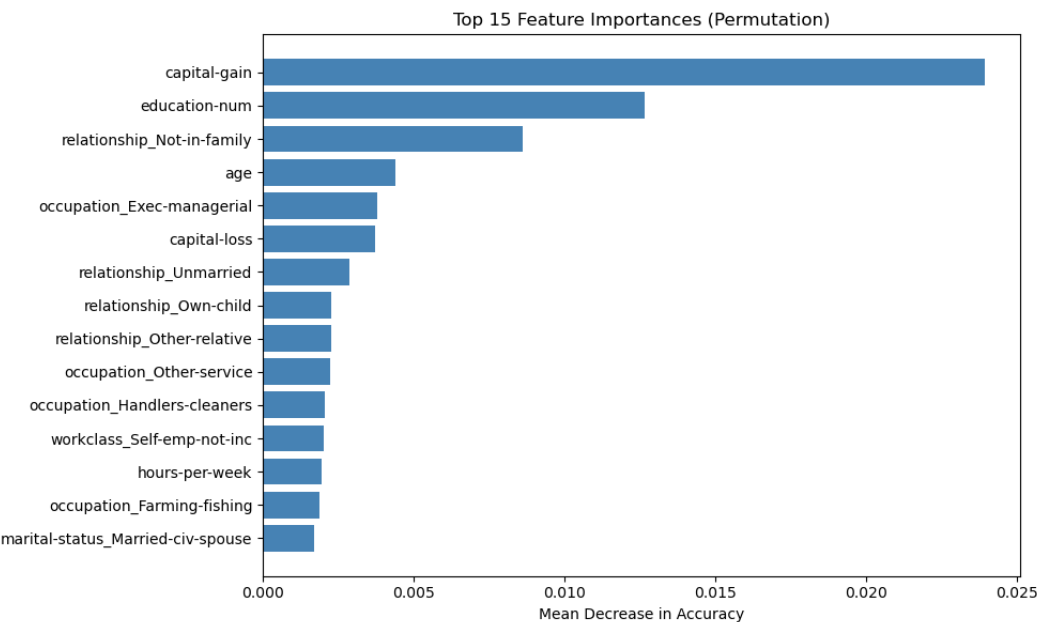


AUC measures the model's ability to correctly rank a randomly chosen high-income citizen above a randomly chosen low-income citizen, with 1.0 representing a perfect classifier and 0.5 representing random chance.

Feature Importance

Because KNN has no internal coefficients or weights, permutation importance was used to assess feature contribution. Each feature was randomly shuffled one at a time, and the resulting drop in model accuracy was recorded. A larger drop indicates greater reliance on that feature.

Capital-gain emerged as the strongest predictor by a considerable margin, with a mean accuracy decrease of approximately 0.025. This suggests that existing wealth and investment activity is the most discriminating signal between income groups. Education-num ranked second, confirming that years of formal schooling remain a strong determinant of earning potential. Relationship type (Not-in-family), age, and executive or managerial occupation rounded out the top five. Marital-status (Married-civ-spouse) appeared near the bottom of the top 15, indicating it adds comparatively less discriminating power once capital activity and education are accounted for.



From a policy standpoint, the dominance of capital-gain over all other features points to a structural dynamic in which existing wealth is the strongest barrier separating income groups. Education remains the most actionable lever available to policymakers, as expanding access to higher education is the intervention most likely to produce measurable improvements in income equity across the population.

Accompanying Model Comparison

Two additional models were trained and evaluated to contextualize KNN performance: a decision tree classifier and a logistic regression model. Table 1 summarizes the results.

Model	Accuracy	AUC
KNN (K = 15)	82.79%	0.8748
Decision Tree	80.29%	0.7412
Logistic Regression	84.24%	0.9008

Logistic regression achieved the highest AUC at 0.9008 and the highest accuracy at 84.24%, suggesting that the income boundary in this dataset is largely linearly separable and that parametric models with explicit feature weighting are well suited to this problem. The decision tree achieved the lowest AUC at 0.7412, indicating that its single-split decision boundaries, while interpretable, do not generalize as effectively to this data. KNN sits between these two, offering strong discriminative performance without the strict linearity assumption of logistic regression.

For stakeholder communication, logistic regression would be the preferred model if interpretability of individual feature coefficients is required. KNN is appropriate when local neighborhood effects are meaningful and when no distributional assumptions can be made about the data. The decision tree offers the highest interpretability through visual decision rules but at a cost to generalization.

Conclusion

This analysis demonstrates that demographic and occupational attributes from U.S. Census data can predict individual income classification with meaningful accuracy. A thorough data cleansing pipeline, including outlier capping, redundancy removal, one-hot encoding, and StandardScaler normalization, was essential to prepare the data for KNN modeling. The optimal $K = 15$ was determined through 5-fold cross-validation, and the resulting model achieved 82.79% accuracy and an AUC of 0.8748 on the held-out test set. The accompanying model comparison revealed that logistic regression achieved the highest AUC at 0.9008, suggesting that linear decision boundaries are well suited to this dataset, while the decision tree lagged at 0.7412. KNN performed competitively overall, offering strong discriminative ability without requiring distributional assumptions about the data.

Feature importance analysis identified capital-gain as the strongest predictor of income classification by a considerable margin, followed by education level, relationship type, and age. The prominence of capital-gain points to a structural dynamic in which existing wealth is the most powerful separator between income groups, reinforcing the advantage of those already in higher earning brackets. Education remains the most policy-actionable predictor, as it is a variable directly addressable through public investment and reform. Future work could address the 75/25 class imbalance through oversampling techniques such as SMOTE, and explore ensemble methods such as Random Forest or Gradient Boosting to improve recall on the minority high-income class.

References

- Cover, T., & Hart, P. (1967). Nearest neighbor pattern classification. *IEEE Transactions on Information Theory*, 13(1), 21–27. <https://doi.org/10.1109/TIT.1967.1053964>
- Dua, D., & Graff, C. (2019). *UCI machine learning repository: Adult data set*. University of California, Irvine, School of Information and Computer Sciences. <https://archive.ics.uci.edu/ml/datasets/adult>
- James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021). *An introduction to statistical learning: With applications in R* (2nd ed.). Springer. <https://doi.org/10.1007/978-1-0716-1418-1>
- McKinney, W. (2010). Data structures for statistical computing in Python. *Proceedings of the 9th Python in Science Conference*, 445, 51–56.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.